ECS 240 Project - Telltale

Rajiv Choudhury, Jacob Hamzawi,
Rishika Nigam, Gurpreet Singh

November 13, 2026

1 Introduction and Motivation

1.1 Problem Domain and Motivation

A function *side effect* is a modification of state that outlives the function, that is, state not wholly contained within the function's scope. We say a function is *pure* if it is free of side effects, and *impure* if it has at least one.

Side effects make code difficult to reason about: at a call site, a reader must inspect the callee and everything it calls to know what it may modify[1]. They can also make for less maintainable and modular code[2,3]. Yet effects are often necessary, as with file I/O and random number generation: how do we allow them while mitigating their downsides?

1.2 Proposed Language

We propose *Telltale*, a toy systems language similar to C and Rust, in which every function is explicitly declared pure or impure. It will include at least variables, functions, references, branches, and 32-bit integer and boolean primitives, and may support structs, strings, loops, and simple file I/O depending on our timeline (see §§ 2 and 4).

Early Rust enforced functions with I/O side effects be marked with an `io` keyword but later removed it[4]; we hope to discover what explicit side effect declarations enable, and whether that decision was justified.

1.3 Proposed Tool

Our main tool is a **compiler** from our language to Rust source code, accompanied by a basic interpreter. The user interacts with both tools from the command line: `tt prog.tt -o prog.rs` compiles a program, while `tt run prog.tt` interprets it directly.

1.4 Scope and Related Work

Tracking side effects is not a new problem: Haskell separates effects from pure code with the monad[5], and functional languages discourage them by design[2]. Effect systems originate with Lucassen and Gifford[6] and Talpin and Jouvelot[7]; Deterministic Parallel Java declares region and field paths in method signatures, close to our declarations[8]. Other tools infer effects in existing Java[1] and Rust[9] code. Our contribution is a small, formally specified imperative language with built-in side effect checks, and an assessment of that design.

2 Goals

Minimum goal. A parser, interpreter, and working compiler to Rust for the core language. The compiler must error when a function modifies outside state without declaring it as `impure` along with that state.

Target goal. The above, along with structs with field-level declarations (e.g. `x.value`; see Appendix B) and file I/O. For I/O, we would explore ways to avoid the `io`-keyword explosion that affected Rust's earlier design.

Reach goal. The above, along with a static analysis tool that could enumerate all state a function may modify, including through the functions it calls, or show the functions that may modify a given variable.

3 Syntax and Semantics

Syntax. Below is a draft formal grammar; Fig. 1 gives example syntax, and Appendix B a fuller annotated example.

The start symbol is ⟨*program*⟩; lexical rules, precedence, and return values are in Appendix B.

⟨*program*⟩ ::= ⟨*decl*⟩ | ⟨*decl*⟩ ⟨*program*⟩
⟨*decl*⟩ ::= `var` ⟨*var*⟩ `;`
| `fn` ⟨*effects*⟩ ⟨*func*⟩`(`⟨*param*⟩$_1$`,` ... `,`⟨*param*⟩$_k$`)` `{` ⟨*body*⟩ `}`
⟨*param*⟩ ::= ⟨*var*⟩ | `&`⟨*var*⟩
⟨*effects*⟩ ::= ϵ | `impure` | `impure(`⟨*place*⟩$_1$`,` ... `,`⟨*place*⟩$_m$`)`
⟨*body*⟩ ::= ⟨*stmts*⟩ | ⟨*stmts*⟩ ⟨*expr*⟩
⟨*stmts*⟩ ::= ⟨*stmt*⟩$_1$ ... ⟨*stmt*⟩$_j$
⟨*stmt*⟩ ::= `var` ⟨*var*⟩ `;` | `var` ⟨*var*⟩ `=` ⟨*expr*⟩ `;` | ⟨*place*⟩ `=` ⟨*expr*⟩ `;`
| ⟨*call*⟩ `;` | `if` ⟨*expr*⟩ `{` ⟨*stmts*⟩ `}`
| `if` ⟨*expr*⟩ `{` ⟨*stmts*⟩ `}` `else` `{` ⟨*stmts*⟩ `}`
⟨*call*⟩ ::= ⟨*func*⟩`(`⟨*expr*⟩$_1$`,` ... `,`⟨*expr*⟩$_k$`)`
⟨*place*⟩ ::= ⟨*var*⟩ | ⟨*place*⟩`.`⟨*field*⟩ | `*`⟨*place*⟩
⟨*expr*⟩ ::= ⟨*int*⟩ | ⟨*bool*⟩ | ⟨*place*⟩ | `&`⟨*place*⟩ | ⟨*call*⟩ | `(`⟨*expr*⟩`)`
| ⟨*unop*⟩ ⟨*expr*⟩ | ⟨*expr*⟩ ⟨*binop*⟩ ⟨*expr*⟩
⟨*bool*⟩ ::= `true` | `false`
⟨*unop*⟩ ::= `-` | `!`
⟨*binop*⟩ ::= `+` | `-` | `*` | `/` | `==` | `!=` | `<` | `<=` | `>` | `>=` | `&&` | `||`

where $k, m, j \geq 0$. A ⟨*place*⟩ names a location that can be read, assigned, or declared in an `impure` clause: a variable, a field of a place (once structs are added), or the target of a reference. `&`⟨*place*⟩ creates a mutable reference (Rust's `&mut`), and a parameter `&`⟨*var*⟩ is passed by reference.

Semantics. The semantics of our language is the set of `(INPUT, prog, OUTPUT)` triples produced by our reference interpreter, which serves as the oracle for the compiler: the emitted Rust must produce the same output. Programs take no input, so `INPUT` is empty (files may become input if we add file I/O). A program that fails the purity check is rejected before it runs and belongs to no triple; otherwise, `OUTPUT` is the final values of all global variables once `main` returns, or an error if execution faults (e.g., integer overflow or division by zero); Appendix B states the purity check's guarantees.

```
fn pure_function(x, y) {
    x + y
}

var z; // global variable

// Declare out-of-scope writes
fn impure(z) add_to_z(x, y) {
    z = x + y;
}
```

```
// main lists all state changes
fn impure main() {
    var x = 1;
    var y = 2;
    x = pure_function(x, y);
    // mutated: z
    add_to_z(x, y);
}
```

Figure 1: Example Telltale program.

4 Implementation Plan

4.1 Tool and Language Selection; Existing Tools

We will build everything from scratch in Rust, which is also our target language.

4.2 Architecture

As shown in Fig. 2 (Appendix C), the parser turns source code into a shared core representation, which the interpreter executes directly and the compiler lowers to an IR, checks for purity, and translates to Rust; both paths report violations as diagnostics.

4.3 Development Effort

We will build each component above: the **core** language definitions, the **parser**, the **interpreter**, and the **compiler**'s lowering, purity checker, and Rust code generator.

4.4 Challenges

Scope tracking and control flow. Tracking mutations through nested calls, references, and branches is complex, and raises whether state that only *might* change must be declared. If this proves too difficult, we will restrict

scopes to globals and immediate parameters and conservatively require declarations for anything a conditional block could modify.

Analysis tool overhead. Robust static analysis across every edge case may exceed our timeline; if so, we will drop the analysis tool (a reach goal).

5 Collaboration Plan

Communication. Slack, GitHub; meet Mon. after class, Thu. on Zoom.

Division of Work and Timeline.

Component	Member(s)
Core	Jacob
Parser	Rishika
Interpreter	Gurpreet, Jacob
Compiler	Rajiv, Jacob
Analysis tool	TBD

Week	Components
Week 1	Parser + Core
Week 2	Interpreter + Compiler
Week 3	Interpreter + Compiler
Week 4	Analysis tool
	Presentation

Commitment statement.
Rajiv Choudhury: I commit to work and contribute equally to this project.
Jacob Hamzawi: I commit to work and contribute equally to this project.
Rishika Nigam: I commit to work and contribute equally to this project.
Gurpreet Singh: I commit to work and contribute equally to this project.

6 Conclusion and Questions for Feedback

Telltale will let us explore whether compiler-checked effect declarations make imperative programs easier to reason about, and weigh their guarantees against the annotation effort they require. A central question is whether tracking mutations to variables and referenced data is enough to evaluate the design, or whether we should also model system state such as file I/O; we would appreciate feedback on whether that broader scope justifies its complexity within our timeline. We would also like feedback on whether Telltale is too similar to Rust, given that Rust is both our implementation and target language.

References Cited

[1] Alexandru Sălcianu and Martin Rinard. Purity and Side Effect Analysis for Java Programs. In Radhia Cousot, David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Madhu Sudan, Demetri Terzopoulos, Dough Tygar, Moshe Y. Vardi, and Gerhard Weikum, editors, *Verification, Model Checking, and Abstract Interpretation*, volume 3385 of *Lecture Notes in Computer Science*, pages 199–215. Springer Berlin Heidelberg, Berlin, Heidelberg, 2005. ISBN 978-3-540-24297-0 978-3-540-30579-8. doi: 10.1007/978-3-540-30579-8_14. URL http://link.springer.com/10.1007/978-3-540-30579-8_14.

[2] J. Hughes. Why Functional Programming Matters. *The Computer Journal*, 32(2):98–107, February 1989. ISSN 0010-4620, 1460-2067. doi: 10.1093/comjnl/32.2.98. URL https://academic.oup.com/comjnl/article-lookup/doi/10.1093/comjnl/32.2.98.

[3] John Backus. Can programming be liberated from the von Neumann style? a functional style and its algebra of programs. *Communications of the ACM*, 21(8):613–641, August 1978. ISSN 0001-0782. doi: 10.1145/359576.359579. URL https://dl.acm.org/doi/10.1145/359576.359579.

[4] Graydon Hoare. Re: Sub-grammar for range pattern constants? rust-dev mailing list, Mozilla, April 2013. URL https://mail-archive.com/rust-dev@mozilla.org/msg03913.html. Accessed: 2026-11-11.

[5] Simon L. Peyton Jones and Philip Wadler. Imperative functional programming. In *Proceedings of the 20th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, POPL '93, pages 71–84, New York, NY, USA, March 1993. Association for Computing Machinery. ISBN 978-0-89791-560-1. doi: 10.1145/158511.158524. URL https://dl.acm.org/doi/10.1145/158511.158524.

[6] J. M. Lucassen and D. K. Gifford. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, POPL '88, pages 47–57, New York, NY, USA, January 1988. Association for Computing Machinery. ISBN 978-0-89791-252-5. doi: 10.1145/73560.73564. URL https://dl.acm.org/doi/10.1145/73560.73564.

[7] J. P. Talpin and P. Jouvelot. The Type and Effect Discipline. *Information and Computation*, 111(2):245–296, June 1994. ISSN 0890-5401. doi: 10.1006/inco.1994.1046. URL https://www.sciencedirect.com/science/article/pii/S0890540184710467.

[8] Robert L. Bocchino, Vikram S. Adve, Danny Dig, Sarita V. Adve, Stephen Heumann, Rakesh Komuravelli, Jeffrey Overbey, Patrick Simmons, Hyojin Sung, and Mohsen Vakilian. A type and effect system for deterministic parallel Java. In *Proceedings of the 24th ACM SIGPLAN conference on Object oriented programming systems languages and applications*, OOPSLA '09, pages 97–116, New York, NY, USA, October 2009. Association for Computing Machinery. ISBN 978-1-60558-766-0. doi: 10.1145/1640089.1640097. URL https://dl.acm.org/doi/10.1145/1640089.1640097.

[9] Lydia Zoghbi, David Thien, Ranjit Jhala, Deian Stefan, and Caleb Stanford. Auditing Rust Crates Effectively. In *Programming Languages and Systems: 35th European Symposium on Programming, ESOP 2026, Held as Part of the International Joint Conferences on Theory and Practice of Software, ETAPS 2026, Turin, Italy, April 11–16, 2026, Proceedings, Part II*, pages 424–444, Berlin, Heidelberg, April 2026. Springer-Verlag. ISBN 978-3-032-22722-5. doi: 10.1007/978-3-032-22723-2_15. URL https://doi.org/10.1007/978-3-032-22723-2_15.

A Introduction and Motivation

Early Rust took our approach, treating functions as pure by default and marking I/O with an `io` keyword[4], but its designers later removed the pure/impure separation.

While Rust removed the `io`, `pred`, and other keywords related to pure/impure separation, it retains `mut` and `const` keywords to define mutability, significantly decreasing the possible surface area of mutation state.

B Syntax and Semantics

$\langle int \rangle$ is a decimal literal from 0 to $2^{31} - 1$ (values are 32-bit signed integers). $\langle var \rangle$, $\langle func \rangle$, and $\langle field \rangle$ are identifiers (letters, digits, and underscores, be-

ginning with a letter) naming variables, functions, and struct fields.

The grammar for ⟨*expr*⟩ is ambiguous as written. Unary `*`, `&`, `-`, and `!` bind tightest, then `*` and `/`, then `+` and `-`, then the comparisons, then `&&`, then `||`. Comparisons are non-associative; all other binary operators are left-associative. The condition of an `if` must evaluate to a boolean. A trailing ⟨*expr*⟩ with no semicolon is the function's return value, as in Rust.
A complete annotated example:

```
fn pure_function(x, y) {
    x + y
}

// global variable
var z;

// You must specify the data out of scope whose
// state may change
fn impure(z) add_to_z(x, y) {
    z = x + y;
}

// Data reached through a reference is out of scope,
// so writing through it must be declared
fn impure(*r) increment(&r) {
    *r = *r + 1;
}

// You can look at the program and see a full list
// of all the things that could change state
fn impure main() {
    var x = 1;
    var y = 2;
    x = pure_function(x, y);
    // mutated: z
    add_to_z(x, y);
    // mutated: x (through the reference)
    increment(&x);
}
```

Field-level declarations for structs (a target goal):

```
// If variables have fields, those fields
// must be declared
fn impure(x.value, y.value) impure_function_struct(x, y) {
    // mutates x
    x.value = 1;
    y.value = 1;
}
```

Purity guarantees. The purity check must guarantee that a call to a function declared `impure`$(p_1, \ldots, p_m)$ changes only $p_1, \ldots, p_m$, and a call to a pure function changes no state outside its scope. The interpreter verifies this by comparing state before and after each call, so any violation exposes a bug in the purity check.

C Architecture

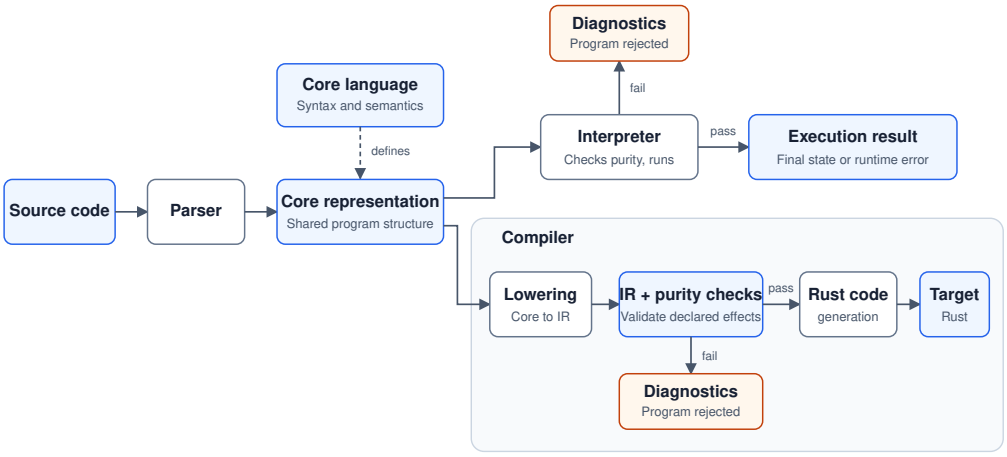


Figure 2: Tool architecture, showing the interpreter and compiler paths from the shared core representation.

1. **Source Code Input:** Raw source string of the custom language.

2. **Parser & AST Builder:** Tokenizes the source text and constructs an Abstract Syntax Tree (AST) representing variables, functions, expressions, and purity annotations.

3. **Interpreter / Evaluator:** Directly evaluates the AST to execute programs for testing and validation, verifying state mutation constraints at runtime.

4. **Compiler & Intermediate Representation (IR):** Transforms the AST into an intermediate representation where strict purity and state-modification checks are performed. It flags compilation errors if an unannotated function attempts out-of-scope mutations.

5. **Target Code Generation:** Translates the validated IR into target Rust source code, which can then be compiled and run natively.